

30. Un personnage de seize

Huit pixels de côté, c'est petit pour un héros.

1. Ce qu'il faut comprendre

Une **Tuile** fait huit pixels de côté. C'est la taille d'une case du décor, et c'est bien pour un mur ou une pièce — mais un personnage y tient mal. Un **Perso** en fait **seize sur seize** : seize rangées de seize signes, écrites comme celles d'une tuile.

Le matériel, lui, ne connaît que des carrés de huit. Un **Perso** occupe donc QUATRE numéros de tuile, dans cet ordre : haut-gauche, haut-droite, bas-gauche, bas-droite. Si **HEROS** vaut 44, il tient les numéros 44, 45, 46 et 47 — et le dessin suivant du programme commence à 48.

`sprite16(n, x, y, HEROS)` pose les **quatre lutins en un seul appel**, en carré, au pixel près. Le prix est dit franchement : il coûte quatre lutins des quarante, là où un `sprite()` n'en coûte qu'un. Une troupe de dix personnages de seize, c'est déjà tout le matériel. `cache16(n)` les ôte tous les quatre.

Sur le fond, il n'y a pas de `poser16()` — une case du décor fait huit pixels, et le personnage en couvre quatre. On pose donc ses quarts un par un, et l'ordre est celui des numéros : `poser(10, 8, HEROS); poser(11, 8, HEROS + 1); poser(10, 9, HEROS + 2); poser(11, 9, HEROS + 3);`. C'est ce que fait le programme ci-dessous, en plus du lutin.

Dans l'atelier, « + **perso 16 x 16** » en ajoute un, et la grille passe à seize sur seize. Le reste ne change pas : on peint, et le programme se réécrit.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1675 octets** de cartouche.

1. UN DESSIN — HEROS

[illegible]

à l'écran

2. UNE VARIABLE — X

```
uint8_t x = 60;
```

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    texte(1, 1, "GAUCHE DROITE");

    /* Le même personnage, POSÉ sur le fond : ses quatre quarts, un par case. */
    poser(10, 8, HEROS);
    poser(11, 8, HEROS + 1);
    poser(10, 9, HEROS + 2);
    poser(11, 9, HEROS + 3);

    while (true) {
        image();

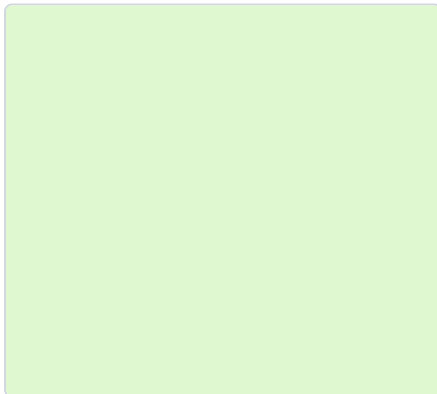
        if (bouton(DROITE) && x < 144) x++;
        if (bouton(GAUCHE) && x > 8) x--;

        /* En lutin : quatre lutins posés en carré, en un seul appel. */
        sprite16(0, x, 100, HEROS);
    }

    return 0;
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



DROITE — l'écran a changé



GAUCHE — l'écran a changé

5. Ce qu'on doit voir

Un héros posé sur le décor, et le même en lutin plus bas — GAUCHE et DROITE le déplacent au pixel près.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le titre est écrit
- ✓ les quatre quarts sont posés sur le fond, dans l'ordre (tuiles 44, 45, 46, 47)
- ✓ `sprite16()` a rempli QUATRE lutins (tuiles 44, 45, 46, 47)
- ✓ ils sont posés en carré
- ✓ DROITE déplace le personnage entier (de 60 à 80)

7. La même chose, à la souris

Cette leçon porte son atelier : le geste réécrit le programme ci-dessus, et la cartouche se refait dans la seconde. C'est le même texte, écrit autrement.



HEROS

+ tuile 8 × 8

+ perso 16 × 16

Game Boy — 4 nuances

Les quatre nuances de la console d'origine, et rien de plus.

Passer en couleur

nuance 3

Nuances

HEROS

tuiles n° 44 à 47 — sprite16(n, x, y, HEROS)

– zoom

11×

+ zoom

Plein écran

Peindre ✓

Sélectionner et déplacer

Copier le dessin

Coller ici

Dupliquer

Effacer le personnage



taille réelle — 16 × 16 pixels

Ensuite : **31. Déplacer un lutin** — niveau 4 sur 10. · gameboy3 — écrire une cartouche Game Boy en C++